

Criptografía — Reporte de Vulnerabilidades (SDDV)

Barrios Aguilar Dulce Michelle
Caballero Martínez Sergio Jair
Contreras Colmenero Emilio Sebastian
Martínez López Evan Emiliano

Facultad de Ingeniería · UNAM 2026-2

Resumen

En el reporte anterior probamos los cinco vectores que pedía la actividad (metadata, lista de destinatarios, nonce, firma y key identifier) y vimos que el sistema los detectaba todos. Pero como nos pidieron ahora reescribir el reporte buscando vulnerabilidades reales con su parche y todo, nos pusimos a revisar el código más a fondo y encontramos dos cosas que sí son problemas de seguridad de verdad. En este reporte explicamos cada una con la estructura que pidió la profe, mostramos cómo se reproducen y los parches que ya subimos al repo.

Las dos vulnerabilidades que encontramos:

ID	Título	CWE	Severidad	Commit del parche
VULN-001	Path Traversal en el filename	CWE-22	ALTA	b40d0c0
VULN-002	Replay Attack (sin freshness)	CWE-294	MEDIA	6a98cd1

VULN-001 — Path Traversal en el filename

Título

Path Traversal en el campo filename de los contenedores SDDV/SDDH (CWE-22)

Descripción

Lo que pasa con esta vulnerabilidad es que cuando el sistema cifra un archivo, el nombre que le pongas se guarda dentro del contenedor sin que nadie revise si es un nombre seguro o no. Y al momento de descifrar, el nombre se devuelve tal cual, sin sanitizar. Es decir, si el atacante mete cosas raras en el nombre, esas cosas raras llegan intactas hasta la aplicación que recibe el archivo.

El problema es que muchas aplicaciones, después de descifrar, hacen algo así para guardar el archivo en disco:

```
plaintext, meta = decrypt_file(container, key)
with open(os.path.join(out_dir, meta['filename']), 'wb') as f:
    f.write(plaintext)
```

Y si el atacante puso como filename algo como `../../../../etc/passwd`, pues ese `os.path.join` no va a guardar en `out_dir` sino que va a salir del directorio destino y va a sobrescribir un archivo del sistema. Y la librería ni se entera, no avisa, no rechaza nada.

Esto afecta tanto al cifrado simétrico (`crypto/aead.py`) como al cifrado híbrido (`crypto/hybrid.py`), porque los dos comparten la misma forma de armar y leer el header del contenedor.

Impacto

Severidad: ALTA (CWE-22)

Qué propiedad de seguridad afecta: principalmente Integridad (porque permite sobrescribir archivos sin que el sistema lo detecte). Indirectamente también puede afectar Confidencialidad y Disponibilidad, dependiendo de qué archivos sobrescriba el atacante.

Quién puede explotar esto: cualquiera que pueda cifrar un contenedor que el sistema vaya a procesar. Esto incluye a usuarios legítimos del sistema con malas intenciones.

Qué puede pasar en la práctica:

- Sobrescribir archivos de configuración del sistema, como `/etc/cron.d/`, `.bashrc` o `.ssh/authorized_keys`
- Si el sistema corre con privilegios, podría llevar a ejecución remota de código (RCE)
- Sobrescribir archivos del usuario sin que se entere
- Engañar al sistema para escribir en cualquier ruta arbitraria del filesystem

Pasos de reproducción

Lo que necesitas para reproducir esto:

- Tener instalado el paquete `cryptography` (`pip install cryptography`)
- Estar en la raíz del repositorio del proyecto
- (Opcional) Hacer checkout al commit anterior al fix para ver la vulnerabilidad sin parchar

Paso 1. Si quieres ver la vulnerabilidad en su versión sin parchar, regresa al commit anterior:

```
git checkout 867516c
```

Paso 2. Corre el script de reproducción que dejamos en `audit/`:

```
PYTHONPATH=. python3 audit/vuln1_path_traversal.py
```

Paso 3. Vas a ver que el sistema acepta sin protestar todos estos filenames maliciosos:

```
[ACEPTADO] filename = '../.../etc/passwd'
            metadata['filename'] retornado = '../.../etc/passwd'
[ACEPTADO] filename = '/etc/shadow'
            metadata['filename'] retornado = '/etc/shadow'
[ACEPTADO] filename = 'valid.txt\x00../.../tmp/pwned'
            metadata['filename'] retornado = 'valid.txt\x00../.../tmp/pwned'
[ACEPTADO] filename = '....//....//etc/secret'
            metadata['filename'] retornado = '....//....//etc/secret'
```

Paso 4. En el script también dejamos un caso donde se demuestra la explotación de verdad. Lo que hace el script es:

1. Crea un directorio destino legítimo en `/tmp/sddv_audit_XXX/`
2. Crea un “archivo crítico” en otro directorio (`/tmp/critical_YYY/config.cfg`) con contenido `contenido_legitimo=true`
3. El atacante cifra un payload con contenido `contenido_malicioso=true` y le pone como filename una ruta relativa que apunta a ese archivo crítico desde el directorio destino: `../critical_YYY/config.cfg`
4. La aplicación vulnerable recibe el contenedor, lo descifra, y hace `open(os.path.join(out_dir, meta['filename']), 'wb').write(plaintext)`
5. **Resultado:** el archivo crítico fue sobrescrito por el contenido del atacante. El sistema no se enteró, todo se hizo pasar por una operación normal.

Paso 5. Si quieres reproducirlo manualmente con menos código:

```
from crypto.aead import encrypt_file, decrypt_file
container, key = encrypt_file(b"payload", "../.../etc/passwd")
plaintext, meta = decrypt_file(container, key)
print(meta['filename'])    # imprime '../.../etc/passwd' sin protestar
```

Solución propuesta

Por qué decidimos arreglarlo así:

Lo primero que pensamos fue: “esto es responsabilidad de la aplicación que usa la librería, no de la librería en sí”. Pero después leímos sobre el principio de *defense-in-depth* y nos dimos cuenta de que una librería de criptografía no debería confiar en que la app que la usa va a sanitizar bien los datos. Si la librería va a aceptar y devolver un filename, lo más responsable es que ella misma valide que sea seguro.

Decidimos validar el filename en dos lugares:

- **Al cifrar:** para que el sistema nunca produzca contenedores con filenames raros
- **Al descifrar:** para proteger casos donde alguien recibe un contenedor que fue cifrado con una versión vieja de la librería que no validaba (*defense-in-depth*)

Lo que rechazamos:

- Separadores de path: `/` y `\`
- Componentes de directorio padre como `..`
- Rutas absolutas (Unix tipo `/etc/...` o Windows tipo `C:`)
- Bytes nulos `\x00` (que se usan para hacer null byte injection)

- Caracteres de control (los que tienen código menor a 0x20)
- Filenames vacíos
- Filenames más largos que 255 bytes (que es el límite estándar de la mayoría de filesystems)

El código del parche (extracto de `crypto/aead.py`):

```
def _validate_filename(filename: str) -> None:
    """Valida que el filename sea seguro para uso en filesystem (CWE-22)."""
    if not isinstance(filename, str):
        raise ValueError("filename debe ser str")
    if not filename:
        raise ValueError("filename no puede estar vacío")
    if len(filename.encode("utf-8")) > MAX_FILENAME_LEN:
        raise ValueError(f"filename excede {MAX_FILENAME_LEN} bytes")
    if "\x00" in filename:
        raise ValueError("filename contiene byte nulo (null byte injection)")
    if any(ord(c) < 0x20 for c in filename):
        raise ValueError("filename contiene caracteres de control")
    if "/" in filename or "\\" in filename:
        raise ValueError("filename contiene separadores de path")
    if filename in (".", ".."):
        raise ValueError("filename no puede ser '.' o '..")
    if len(filename) >= 2 and filename[1] == ":":
        raise ValueError("filename parece ruta absoluta de Windows")
    if posixpath.normpath(filename) != filename:
        raise ValueError("filename contiene componentes de path no canonicos")
```

Commit del parche: b40d0c0 ([feature/d3-hybrid en GitHub](#))

Lo que cambia el parche:

- `crypto/aead.py`: agregamos la función `_validate_filename` y la invocamos al cifrar y al descifrar
- `crypto/hybrid.py`: importamos la función y la invocamos también en el cifrado híbrido
- `tests/test_security_patches.py`: agregamos 16 tests de regresión que verifican que todos los filenames maliciosos sean rechazados y que los nombres normales sigan funcionando

Cómo verificar que el parche funciona: después de aplicar el fix, si corres el script `audit/vuln1_path_traversal.py` ya no vas a ver [ACEPTADO] en los nombres maliciosos sino [RECHAZADO] `ValueError: filename contiene separadores de path`. Y los 16 tests de regresión pasan: `pytest tests/test_security_patches.py::TestVuln001PathTraversal -v`.

VULN-002 — Replay Attack (sin freshness)

Título

Falta de validación de freshness en el timestamp permite replay attacks (CWE-294)

Descripción

Esta vulnerabilidad es más sutil que la anterior. El sistema guarda un campo `timestamp` (la hora Unix de cuando se cifró) dentro del AAD del contenedor. Como está en el AAD, el timestamp queda autenticado: si alguien intenta modificarlo, se invalida el TAG y el sistema lo detecta. **Hasta ahí, todo bien.**

El problema es que el sistema **nunca compara ese timestamp contra la hora actual**. Solo lo guarda y lo devuelve, pero no le pregunta “¿este mensaje es de hace cuánto?”. Lo que significa que un contenedor firmado en 2020 sigue siendo aceptado como válido en 2026, en 2030, en 2099. Para siempre.

¿Por qué es un problema? Porque un atacante que pueda capturar un contenedor legítimo (digamos, sniffando el canal donde se transmite, o copiándolo del storage) puede después reentregarlo tiempo después y hacerlo pasar como si fuera nuevo. La firma sigue siendo válida porque Ed25519 es determinista (los mismos bytes producen la misma firma siempre), y como el contenido no cambió, todo se ve perfecto desde el lado del sistema.

Un detalle importante: el atacante **no puede modificar el timestamp** para hacer parecer que el mensaje es nuevo. Si lo intenta, el TAG falla y se detecta. Pero **tampoco lo necesita**: con que el sistema no valide la edad del mensaje, ya con eso le basta para reproducir el ataque.

Impacto

Severidad: MEDIA (CWE-294)

Qué propiedad de seguridad afecta: Autenticidad y frescura. El sistema garantiza que el mensaje fue dicho por quien dice haberlo dicho, pero no garantiza que sea un mensaje reciente. La confidencialidad y la integridad del mensaje siguen intactas.

Quién puede explotar esto: cualquiera con acceso pasivo al canal donde se transmiten los contenedores, o con acceso al storage donde se guardan. No necesita romper criptografía ni tener llaves privadas.

Qué puede pasar en la práctica:

- Re-ejecutar comandos firmados antiguos (transferencias bancarias, autorizaciones, votos electrónicos)
- Saltarse revocaciones: si Alice quitó la autorización, los mensajes anteriores siguen siendo válidos
- Reenviar credenciales o tokens si se transmiten en este formato
- El sistema acepta también timestamps imposibles, como del año 2099, sin protestar

Por qué la severidad no es alta: para que el ataque funcione, el atacante necesita primero capturar un mensaje legítimo (no es un ataque que se pueda hacer “de cero”). Además, el contenido del mensaje no se le revela al atacante (la confidencialidad sigue intacta).

Pasos de reproducción

Lo que necesitas: lo mismo que para VULN-001.

Paso 1. Para ver la vulnerabilidad sin parchar, regresa al commit anterior al fix de VULN-002:

```
git checkout b40d0c0 # VULN-001 ya parchada, pero VULN-002 todavía presente
```

Paso 2. Corre el script de reproducción:

```
PYTHONPATH=. python3 audit/vuln2_replay.py
```

Paso 3. El script muestra tres casos distintos. Te explicamos cada uno:

Caso A — Replay simple sobre cifrado simétrico:

Alice cifra un mensaje con timestamp del 1 de enero de 2020. El atacante captura el contenedor. Tiempo después, en 2026, el atacante reentrega el mismo contenedor a Bob. Bob lo descifra:

```
contenido recuperado: b'Transferir $1000 de Alice a Bob'
edad del mensaje: 2317 días
```

El sistema acepta sin protestar un mensaje que tiene más de seis años. Si esto fuera una orden de transferencia bancaria, ya estaríamos en problemas.

Caso B — Replay sobre contenedor firmado completo (D5):

Este es el caso más realista. Alice firma una autorización con su llave Ed25519 en junio de 2024. El atacante captura el contenedor pasando por la red. En 2026 lo reentrega a Bob, quien usa `secure_verify_and_decrypt` para verificar. La firma pasa, el descifrado funciona, Bob recibe `b'Autorizo la transferencia de fondos. - Alice'` como si fuera reciente. **689 días de antigüedad, ignorados completamente.**

Caso C — Aceptación de timestamps imposibles:

Un cliente con el reloj mal configurado (o un atacante a propósito) puede generar contenedores con timestamp del año 2099. El sistema los acepta sin distinguir entre tiempos posibles e imposibles.

Paso 4. Para reproducir la idea principal manualmente:

```
import time
from crypto.aead import encrypt_file, decrypt_file

# Atacante captura contenedor antiguo (timestamp del 2020)
container, key = encrypt_file(b"comando antiguo", "cmd.txt", timestamp=1577836800)

# Sistema lo acepta hoy sin protestar
plaintext, meta = decrypt_file(container, key)
print(plaintext, "edad:", (time.time() - meta['timestamp']) / 86400, "días")
```

Solución propuesta

Cómo decidimos arreglarlo:

Lo estándar para protegerse contra replay attacks es validar que el timestamp del mensaje esté dentro de una **ventana de tiempo aceptable**. Por ejemplo: rechazar mensajes más viejos que 7 días. Las decisiones que tomamos:

1. **Default seguro:** la validación está activa por defecto con una ventana de 7 días. Eso significa que el comportamiento por default ya es seguro contra replay sin que la app que usa la librería tenga que hacer nada extra.
2. **Tolerancia a desfases de reloj:** los relojes de las computadoras a veces están un poco mal sincronizados. Permitimos hasta 5 minutos de timestamp en el futuro, porque más allá de eso ya es sospechoso. Esto se llama “clock skew tolerance”.

3. **Sin necesidad de mantener estado:** la validación es solo función del timestamp del contenedor y la hora actual. No necesitamos guardar una lista de nonces vistos ni nada por el estilo, lo cual sería más complicado y propenso a errores.
4. **Compatibilidad hacia atrás:** si alguien tiene contenedores viejos de un archivo histórico que necesita poder abrir, puede pasar `max_age_seconds=None` para deshabilitar la validación de manera explícita. No queríamos romper casos legítimos donde alguien quiere abrir mensajes viejos a propósito.

El código del parche (extracto de `crypto/aead.py`):

```
DEFAULT_MAX_AGE = 7 * 24 * 60 * 60    # 7 días por defecto
MAX_FUTURE_SKEW = 5 * 60              # 5 minutos de tolerancia

def _validate_timestamp(timestamp, max_age_seconds):
    if max_age_seconds is None:
        return
    now = int(time.time())
    age = now - timestamp
    if age > max_age_seconds:
        raise ValueError(f"timestamp demasiado antiguo: {age} segundos "
                        f"(max permitido: {max_age_seconds})")
    if age < -MAX_FUTURE_SKEW:
        raise ValueError(f"timestamp en el futuro: {-age} segundos "
                        f"(max skew permitido: {MAX_FUTURE_SKEW})")

def decrypt_file(container, key, max_age_seconds=DEFAULT_MAX_AGE):
    metadata, header_end = _parse_header(container)
    _validate_timestamp(metadata["timestamp"], max_age_seconds)
    # ... resto del flujo de descifrado ...
```

Commit del parche: 6a98cd1 ([feature/d3-hybrid](#) en GitHub)

Lo que cambia el parche:

- `crypto/aead.py`: agregamos la función `_validate_timestamp`, las constantes `DEFAULT_MAX_AGE` y `MAX_FUTURE_SKEW`, y el parámetro nuevo `max_age_seconds` en `decrypt_file`
- `crypto/hybrid.py`: agregamos el mismo parámetro en `decrypt_for_recipient`
- `crypto/secure_send.py`: pasamos el parámetro a través de `secure_verify_and_decrypt`
- `tests/test_aead.py` y `test_hybrid.py`: tuvimos que actualizar dos tests existentes que usaban timestamps del pasado para que pasaran `max_age_seconds=None` (porque están probando otra cosa, no la freshness)
- `tests/test_security_patches.py`: agregamos 8 tests de regresión que cubren todos los casos (timestamp viejo rechazado, futuro rechazado, ventana válida aceptada, clock skew tolerado, etc.)

Cómo verificar que el parche funciona: después de aplicar el fix, si corres `audit/vuln2_replay.py` ahora ves `ValueError: timestamp demasiado antiguo: 200177807 segundos (max permitido: 604800)` en el caso A, y errores similares en los otros casos. Los 8 tests de regresión nuevos pasan, y los 122 tests originales del proyecto siguen pasando sin cambios.

Verificación de que todo sigue funcionando

Tests

Después de aplicar los dos parches, corrimos toda la suite de tests para asegurarnos de que no rompimos nada del sistema:

```
$ PYTHONPATH=. python3 -m pytest tests/
===== test session starts =====
collected 146 items

tests/test_aead.py ..... [ 18%]
tests/test_d5_hybrid_signed.py ..... [ 38%]
tests/test_hybrid.py ..... [ 59%]
tests/test_keys.py ..... [ 71%]
tests/test_security_patches.py ..... [ 88%]
tests/test_signatures.py ..... [100%]

===== 146 passed in 0.27s =====
```

Lo que esto nos dice:

- Los 122 tests que ya teníamos antes siguen pasando, así que los parches no rompieron nada del comportamiento original.
- Los 24 tests nuevos que agregamos para cubrir las dos vulnerabilidades pasan también:
 - 16 tests para VULN-001 (path traversal en cifrado simétrico, en cifrado híbrido, defense-in-depth, casos válidos que deben seguir funcionando)
 - 8 tests para VULN-002 (rechazo de timestamps antiguos, futuros, ventana válida, clock skew, max_age personalizado, parche aplicado en todas las funciones de descifrado)

Estado del repositorio

Para que se pueda verificar todo, dejamos los commits separados así:

```
$ git log --oneline -3
6a98cd1 fix(VULN-002): validar freshness de timestamp para prevenir replay
b40d0c0 fix(VULN-001): validar filename para prevenir path traversal
867516c feat: agregar auditoria de seguridad - script de tampering, log y reporte PDF
```

Los dos commits están en la rama `feature/d3-hybrid` y se pueden ver directo en GitHub:

- **VULN-001:** <https://github.com/milliyx/Cryptography/commit/b40d0c0>
- **VULN-002:** <https://github.com/milliyx/Cryptography/commit/6a98cd1>

Conclusión

Las dos vulnerabilidades que encontramos nos dejaron un par de aprendizajes importantes:

De VULN-001 (path traversal) aprendimos que el cripto por sí solo no es suficiente. El sistema cumple su trabajo de proteger la confidencialidad y la integridad del contenido, pero si la metadata se devuelve sin sanitizar, esa misma metadata puede convertirse en el vector de ataque. Es como tener una caja fuerte muy buena pero con la etiqueta exterior escrita por el atacante. Por eso el principio de defense-in-depth es importante: hay que poner controles en todas las capas, no solo donde están las primitivas criptográficas.

De VULN-002 (replay) aprendimos algo que al principio nos sonaba raro: una firma válida no significa que el mensaje sea reciente. Una firma garantiza “esto fue dicho por X” pero no “esto fue dicho ahorita”. Si tu sistema necesita que los mensajes sean recientes (porque, por ejemplo, son comandos o autorizaciones que podrían cambiar de validez con el tiempo), tienes que agregar la protección anti-replay tú mismo. La criptografía no te lo hace gratis.

Las dos vulnerabilidades las pudimos arreglar con cambios chicos: un par de funciones helper y unas pocas líneas en los puntos de entrada. Los parches mantienen compatibilidad hacia atrás (puedes deshabilitar las validaciones nuevas si las necesitas para casos especiales), no afectan el rendimiento del sistema, y vienen con tests de regresión que confirman que cierran las vulnerabilidades sin romper nada más.